

Programmation événementielle en Python

La bibliothèque Tkinter

Aurel Megnigbeto

Université d'Abomey-Calavi

École Nationale d'Économie Appliquée et de Management

29 février 2024

Plan

- 1 Pourquoi Tkinter
- 2 Historique
- 3 Installation
- 4 Fonctionnement
 - Concepts de base
 - Widgets
 - Frame
 - Label

Plan

- 1 Pourquoi Tkinter
- 2 Historique
- 3 Installation
- 4 Fonctionnement
 - Concepts de base
 - Widgets
 - Frame
 - Label

Pourquoi Tkinter

Il existe plusieurs bibliothèques de programmation graphique en Python dont :

- PyQt
- Kivy
- wxPython
- Tkinter

On étudie Tkinter car :

- Elle est livrée avec Python
- Elle est simple à utiliser
- Elle est bien documentée
- Elle est multi-plateforme

Pourquoi Tkinter

Il existe plusieurs bibliothèques de programmation graphique en Python dont :

- PyQt
- Kivy
- wxPython
- Tkinter

On étudie Tkinter car :

- Elle est livrée avec Python
- Elle est simple à utiliser
- Elle est bien documentée
- Elle est multi-plateforme

Pourquoi Tkinter

Il existe plusieurs bibliothèques de programmation graphique en Python dont :

- PyQt
- Kivy
- wxPython
- Tkinter

On étudie Tkinter car :

- Elle est livrée avec Python
- Elle est simple à utiliser
- Elle est bien documentée
- Elle est multi-plateforme

Pourquoi Tkinter

Il existe plusieurs bibliothèques de programmation graphique en Python dont :

- PyQt
- Kivy
- wxPython
- Tkinter

On étudie Tkinter car :

- Elle est livrée avec Python
- Elle est simple à utiliser
- Elle est bien documentée
- Elle est multi-plateforme

Pourquoi Tkinter

Il existe plusieurs bibliothèques de programmation graphique en Python dont :

- PyQt
- Kivy
- wxPython
- Tkinter

On étudie Tkinter car :

- Elle est livrée avec Python
- Elle est simple à utiliser
- Elle est bien documentée
- Elle est multi-plateforme

Pourquoi Tkinter

Il existe plusieurs bibliothèques de programmation graphique en Python dont :

- PyQt
- Kivy
- wxPython
- Tkinter

On étudie Tkinter car :

- Elle est livrée avec Python
- Elle est simple à utiliser
- Elle est bien documentée
- Elle est multi-plateforme

Pourquoi Tkinter

Il existe plusieurs bibliothèques de programmation graphique en Python dont :

- PyQt
- Kivy
- wxPython
- Tkinter

On étudie Tkinter car :

- Elle est livrée avec Python
- Elle est simple à utiliser
- Elle est bien documentée
- Elle est multi-plateforme

Pourquoi Tkinter

Il existe plusieurs bibliothèques de programmation graphique en Python dont :

- PyQt
- Kivy
- wxPython
- Tkinter

On étudie Tkinter car :

- Elle est livrée avec Python
- Elle est simple à utiliser
- Elle est bien documentée
- Elle est multi-plateforme

Plan

- 1 Pourquoi Tkinter
- 2 Historique
- 3 Installation
- 4 Fonctionnement
 - Concepts de base
 - Widgets
 - Frame
 - Label

Historique de Tk et Tkinter

Brève historique de Tk :

- Tk est une bibliothèque graphique pour le langage de programmation Tcl.
- Créée par John Ousterhout en 1988.
- Tk a été développé principalement pour être utilisé avec des langages de script (Python, Tcl, Perl, ...)
- La dernière version de Tk est la version 8.6.13 sortie en 2022.

Brève historique de Tkinter :

- Tkinter est l'interface entre Tk et Python.
- Tkinter a été créé par Fredrik Lundh en 1991.
- La dernière version de Tkinter est celle incluse dans Python 3.12

Historique de Tk et Tkinter

Brève historique de Tk :

- Tk est une bibliothèque graphique pour le langage de programmation Tcl.
- Créée par John Ousterhout en 1988.
- Tk a été développé principalement pour être utilisé avec des langages de script (Python, Tcl, Perl, ...)
- La dernière version de Tk est la version 8.6.13 sortie en 2022.

Brève historique de Tkinter :

- Tkinter est l'interface entre Tk et Python.
- Tkinter a été créé par Fredrik Lundh en 1991.
- La dernière version de Tkinter est celle incluse dans Python 3.12

Historique de Tk et Tkinter

Brève historique de Tk :

- Tk est une bibliothèque graphique pour le langage de programmation Tcl.
- Créée par John Ousterhout en 1988.
- Tk a été développé principalement pour être utilisé avec des langages de script (Python, Tcl, Perl, ...)
- La dernière version de Tk est la version 8.6.13 sortie en 2022.

Brève historique de Tkinter :

- Tkinter est l'interface entre Tk et Python.
- Tkinter a été créé par Fredrik Lundh en 1991.
- La dernière version de Tkinter est celle incluse dans Python 3.12

Historique de Tk et Tkinter

Brève historique de Tk :

- Tk est une bibliothèque graphique pour le langage de programmation Tcl.
- Créée par John Ousterhout en 1988.
- Tk a été développé principalement pour être utilisé avec des langages de script (Python, Tcl, Perl, ...)
- La dernière version de Tk est la version 8.6.13 sortie en 2022.

Brève historique de Tkinter :

- Tkinter est l'interface entre Tk et Python.
- Tkinter a été créé par Fredrik Lundh en 1991.
- La dernière version de Tkinter est celle incluse dans Python 3.12

Historique de Tk et Tkinter

Brève historique de Tk :

- Tk est une bibliothèque graphique pour le langage de programmation Tcl.
- Créée par John Ousterhout en 1988.
- Tk a été développé principalement pour être utilisé avec des langages de script (Python, Tcl, Perl, ...)
- La dernière version de Tk est la version 8.6.13 sortie en 2022.

Brève historique de Tkinter :

- Tkinter est l'interface entre Tk et Python.
- Tkinter a été créé par Fredrik Lundh en 1991.
- La dernière version de Tkinter est celle incluse dans Python 3.12

Historique de Tk et Tkinter

Brève historique de Tk :

- Tk est une bibliothèque graphique pour le langage de programmation Tcl.
- Créée par John Ousterhout en 1988.
- Tk a été développé principalement pour être utilisé avec des langages de script (Python, Tcl, Perl, ...)
- La dernière version de Tk est la version 8.6.13 sortie en 2022.

Brève historique de Tkinter :

- Tkinter est l'interface entre Tk et Python.
- Tkinter a été créé par Fredrik Lundh en 1991.
- La dernière version de Tkinter est celle incluse dans Python 3.12

Historique de Tk et Tkinter

Brève historique de Tk :

- Tk est une bibliothèque graphique pour le langage de programmation Tcl.
- Créée par John Ousterhout en 1988.
- Tk a été développé principalement pour être utilisé avec des langages de script (Python, Tcl, Perl, ...)
- La dernière version de Tk est la version 8.6.13 sortie en 2022.

Brève historique de Tkinter :

- Tkinter est l'interface entre Tk et Python.
- Tkinter a été créé par Fredrik Lundh en 1991.
- La dernière version de Tkinter est celle incluse dans Python 3.12

Plan

- 1 Pourquoi Tkinter
- 2 Historique
- 3 Installation**
- 4 Fonctionnement
 - Concepts de base
 - Widgets
 - Frame
 - Label

Installer Thinker

Tkinter est livrée avec Python, mais parfois il faut installer des paquets supplémentaires pour l'utiliser.

```
# !\ Make sure to specify the correct Python version.  
# For example, my Python v is 3.11, so I would install as  
sudo apt-get install python3.11-tk
```

Ressources supplémentaires :

- Pour ceux qui sont sur windows
<https://bobbyhadz.com/blog/python-no-module-named-tkinter>
- Doc officielle de Tkinter
<https://docs.python.org/3/library/tkinter.html>

Installer Thinker

Tkinter est livrée avec Python, mais parfois il faut installer des paquets supplémentaires pour l'utiliser.

```
# !\ Make sure to specify the correct Python version.  
# For example, my Python v is 3.11, so I would install as  
sudo apt-get install python3.11-tk
```

Ressources supplémentaires :

- Pour ceux qui sont sur windows
<https://bobbyhadz.com/blog/python-no-module-named-tkinter>
- Doc officielle de Tkinter
<https://docs.python.org/3/library/tkinter.html>

Installer Thinker

Tester l'installation

Petit script pour tester le bon fonctionnement de Tkinter après l'installation.

```
import tkinter  
  
tkinter._test()
```

Installer Thinker

Hello world

Notre premier "Hello World".

```
from tkinter import *  
from tkinter import ttk  
root = Tk()  
ttk.Button(root, text="Hello World").grid()  
root.mainloop()
```


Plan

- 1 Pourquoi Tkinter
- 2 Historique
- 3 Installation
- 4 **Fonctionnement**
 - Concepts de base
 - Widgets
 - Frame
 - Label

Plan

- 1 Pourquoi Tkinter
- 2 Historique
- 3 Installation
- 4 **Fonctionnement**
 - Concepts de base
 - Widgets
 - Frame
 - Label

Concepts de base

La librairie Tkinter repose sur 3 catégories d'éléments de base :

Widgets Les éléments graphiques de base qu'on peut utiliser pour construire une interface graphique.

Gestionnaires de positionnement Les gestionnaires de positionnement permettent de placer les widgets dans une fenêtre.

Boucle d'évènements La boucle d'évènements est une boucle qui attend les évènements et les traite.

Concepts de base

La librairie Tkinter repose sur 3 catégories d'éléments de base :

Widgets Les éléments graphiques de base qu'on peut utiliser pour construire une interface graphique.

Gestionnaires de positionnement Les gestionnaires de positionnement permettent de placer les widgets dans une fenêtre.

Boucle d'évènements La boucle d'évènements est une boucle qui attend les évènements et les traite.

Concepts de base

La librairie Tkinter repose sur 3 catégories d'éléments de base :

Widgets Les éléments graphiques de base qu'on peut utiliser pour construire une interface graphique.

Gestionnaires de positionnement Les gestionnaires de positionnement permettent de placer les widgets dans une fenêtre.

Boucle d'évènements La boucle d'évènements est une boucle qui attend les évènements et les traite.

Plan

- 1 Pourquoi Tkinter
- 2 Historique
- 3 Installation
- 4 Fonctionnement**
 - Concepts de base
 - **Widgets**
 - Frame
 - Label

Widgets

Les widgets sont les éléments graphiques de base qu'on peut utiliser pour construire une interface graphique. Les widgets les plus courants sont :

- Frame
- Label
- Button
- Checkbutton
- Entry
- Combobox
- ...

Widgets

Les widgets sont les éléments graphiques de base qu'on peut utiliser pour construire une interface graphique. Les widgets les plus courants sont :

- Frame
- Label
- Button
- Checkbutton
- Entry
- Combobox
- ...

Widgets

Les widgets sont les éléments graphiques de base qu'on peut utiliser pour construire une interface graphique. Les widgets les plus courants sont :

- Frame
- Label
- Button
- Checkbutton
- Entry
- Combobox
- ...

Widgets

Les widgets sont les éléments graphiques de base qu'on peut utiliser pour construire une interface graphique. Les widgets les plus courants sont :

- Frame
- Label
- Button
- Checkbutton
- Entry
- Combobox
- ...

Widgets

Les widgets sont les éléments graphiques de base qu'on peut utiliser pour construire une interface graphique. Les widgets les plus courants sont :

- Frame
- Label
- Button
- Checkbutton
- Entry
- Combobox
- ...

Widgets

Les widgets sont les éléments graphiques de base qu'on peut utiliser pour construire une interface graphique. Les widgets les plus courants sont :

- Frame
- Label
- Button
- Checkbutton
- Entry
- Combobox

• . . .

Widgets

Les widgets sont les éléments graphiques de base qu'on peut utiliser pour construire une interface graphique. Les widgets les plus courants sont :

- Frame
- Label
- Button
- Checkbutton
- Entry
- Combobox
- ...

Widgets

Tkinter utilise une approche orientée objet pour la représentation des widgets.

- Avec Tkinter, un widget est une simple classe Python.
- Par conséquent, la création d'un widget se fait par instantiation de la classe correspondante.
- Les widgets ont des propriétés partagées et des propriétés spécifiques.
 - ▶ *Button(root, text = " Cliquez – moi" , command = callback)*
 - ▶ *Label(root, text = " Bonjour")*

Exemple de configuration d'un widget (à faire interactivement).

Widgets

Tkinter utilise une approche orientée objet pour la représentation des widgets.

- Avec Tkinter, un widget est une simple classe Python.
- Par conséquent, la création d'un widget se fait par instantiation de la classe correspondante.
- Les widgets ont des propriétés partagées et des propriétés spécifiques.

▶ *Button(root, text = " Cliquez – moi", command = callback)*

▶ *Label(root, text = " Bonjour")*

Exemple de configuration d'un widget (à faire interactivement).

Widgets

Tkinter utilise une approche orientée objet pour la représentation des widgets.

- Avec Tkinter, un widget est une simple classe Python.
- Par conséquent, la création d'un widget se fait par instanciation de la classe correspondante.
- Les widgets ont des propriétés partagées et des propriétés spécifiques.
 - ▶ *Button(root, text = " Cliquez – moi" , command = callback)*
 - ▶ *Label(root, text = " Bonjour")*

Exemple de configuration d'un widget (à faire interactivement).

Widgets

Tkinter utilise une approche orientée objet pour la représentation des widgets.

- Avec Tkinter, un widget est une simple classe Python.
- Par conséquent, la création d'un widget se fait par instantiation de la classe correspondante.
- Les widgets ont des propriétés partagées et des propriétés spécifiques.
 - ▶ *Button(root, text = " Cliquez – moi" , command = callback)*
 - ▶ *Label(root, text = " Bonjour")*

Exemple de configuration d'un widget (à faire interactivement).

Widgets

Tkinter utilise une approche orientée objet pour la représentation des widgets.

- Avec Tkinter, un widget est une simple classe Python.
- Par conséquent, la création d'un widget se fait par instantiation de la classe correspondante.
- Les widgets ont des propriétés partagées et des propriétés spécifiques.
 - ▶ `Button(root, text = " Cliquez – moi" , command = callback)`
 - ▶ `Label(root, text = " Bonjour" )`

Exemple de configuration d'un widget (à faire interactivement).

Widgets

Exemple de configuration

```
from tkinter import *
from tkinter import ttk
root = Tk()
# create a button, passing two options:
button = ttk.Button(root, text="Hello", command="buttonpressed")
button.grid()
# check the current value of the text option:
button['text']
# change the value of the text option:
button['text'] = 'goodbye'
# another way to do the same thing:
button.configure(text='goodbye')
# check the current value of the text option:
button['text']
# get information on all options for this widget:
button.configure()
```

Widgets

Frame

Un Frame est un conteneur pour d'autres widgets. Il est utilisé pour organiser les widgets dans une fenêtre.

Quelques propriétés :

- padding : marge intérieure
- borderwidth : largeur de la bordure
- relief : type de bordure
- width : largeur
- height : hauteur
- style : style

Widgets

Frame

Un Frame est un conteneur pour d'autres widgets. Il est utilisé pour organiser les widgets dans une fenêtre.

Quelques propriétés :

- padding : marge intérieure
- borderwidth : largeur de la bordure
- relief : type de bordure
- width : largeur
- height : hauteur
- style : style

Widgets

Frame

Un Frame est un conteneur pour d'autres widgets. Il est utilisé pour organiser les widgets dans une fenêtre.

Quelques propriétés :

- padding : marge intérieure
- borderwidth : largeur de la bordure
- relief : type de bordure
- width : largeur
- height : hauteur
- style : style

Widgets

Frame

Un Frame est un conteneur pour d'autres widgets. Il est utilisé pour organiser les widgets dans une fenêtre.

Quelques propriétés :

- padding : marge intérieure
- borderwidth : largeur de la bordure
- relief : type de bordure
- width : largeur
- height : hauteur
- style : style

Widgets

Frame

Un Frame est un conteneur pour d'autres widgets. Il est utilisé pour organiser les widgets dans une fenêtre.

Quelques propriétés :

- padding : marge intérieure
- borderwidth : largeur de la bordure
- relief : type de bordure
- width : largeur
- height : hauteur
- style : style

Widgets

Frame

Un Frame est un conteneur pour d'autres widgets. Il est utilisé pour organiser les widgets dans une fenêtre.

Quelques propriétés :

- padding : marge intérieure
- borderwidth : largeur de la bordure
- relief : type de bordure
- width : largeur
- height : hauteur
- style : style

Widgets

Frame

Exemple de création et configuration d'un Frame :

```
from tkinter import *
from tkinter import ttk

root = Tk()
f = ttk.Frame(root)

f['padding'] = 5 # 5 pixels on all sides
f['padding'] = (5,10) # 5 on left and right, 10 on top and bottom
f['padding'] = (5,7,10,12) # left: 5, top: 7, right: 10, bottom: 12

f['borderwidth'] = 2
f['relief'] = 'sunken'

s = ttk.Style()
s.configure('Danger.TFrame', background='red', borderwidth=5,
relief='raised')
ttk.Frame(root, width=200, height=200,
style='Danger.TFrame').grid()
```

Widgets

Label

- Widget pour afficher du texte ou une image.
- Il ne peut pas être modifié par l'utilisateur.
- On utilise la classe `ttk.Label`
- On peut lier le texte à afficher à une variable.

Quelques propriétés :

- `text` : texte à afficher
- `font` : police de caractères
- `background` : couleur de fond
- `foreground` : couleur du texte
- `relief` : type de bordure

Widgets

Label

- Widget pour afficher du texte ou une image.
- Il ne peut pas être modifié par l'utilisateur.
- On utilise la classe `ttk.Label`
- On peut lier le texte à afficher à une variable.

Quelques propriétés :

- `text` : texte à afficher
- `font` : police de caractères
- `background` : couleur de fond
- `foreground` : couleur du texte
- `relief` : type de bordure

Widgets

Label

- Widget pour afficher du texte ou une image.
- Il ne peut pas être modifié par l'utilisateur.
- On utilise la classe `ttk.Label`
- On peut lier le texte à afficher à une variable.

Quelques propriétés :

- `text` : texte à afficher
- `font` : police de caractères
- `background` : couleur de fond
- `foreground` : couleur du texte
- `relief` : type de bordure

Widgets

Label

- Widget pour afficher du texte ou une image.
- Il ne peut pas être modifié par l'utilisateur.
- On utilise la classe `ttk.Label`
- On peut lier le texte à afficher à une variable.

Quelques propriétés :

- `text` : texte à afficher
- `font` : police de caractères
- `background` : couleur de fond
- `foreground` : couleur du texte
- `relief` : type de bordure

Widgets

Label

- Widget pour afficher du texte ou une image.
- Il ne peut pas être modifié par l'utilisateur.
- On utilise la classe `ttk.Label`
- On peut lier le texte à afficher à une variable.

Quelques propriétés :

- `text` : texte à afficher
- `font` : police de caractères
- `background` : couleur de fond
- `foreground` : couleur du texte
- `relief` : type de bordure

Widgets

Label

- Widget pour afficher du texte ou une image.
- Il ne peut pas être modifié par l'utilisateur.
- On utilise la classe `ttk.Label`
- On peut lier le texte à afficher à une variable.

Quelques propriétés :

- `text` : texte à afficher
- `font` : police de caractères
- `background` : couleur de fond
- `foreground` : couleur du texte
- `relief` : type de bordure

Widgets

Label

- Widget pour afficher du texte ou une image.
- Il ne peut pas être modifié par l'utilisateur.
- On utilise la classe `ttk.Label`
- On peut lier le texte à afficher à une variable.

Quelques propriétés :

- `text` : texte à afficher
- `font` : police de caractères
- `background` : couleur de fond
- `foreground` : couleur du texte
- `relief` : type de bordure

Widgets

Label

- Widget pour afficher du texte ou une image.
- Il ne peut pas être modifié par l'utilisateur.
- On utilise la classe `ttk.Label`
- On peut lier le texte à afficher à une variable.

Quelques propriétés :

- `text` : texte à afficher
- `font` : police de caractères
- `background` : couleur de fond
- `foreground` : couleur du texte
- `relief` : type de bordure

Widgets

Label

- Widget pour afficher du texte ou une image.
- Il ne peut pas être modifié par l'utilisateur.
- On utilise la classe `ttk.Label`
- On peut lier le texte à afficher à une variable.

Quelques propriétés :

- `text` : texte à afficher
- `font` : police de caractères
- `background` : couleur de fond
- `foreground` : couleur du texte
- `relief` : type de bordure

Widgets

Label

Exemple de création et configuration d'un Label :

```
from tkinter import *
from tkinter import ttk

root = Tk()

# Instancier un objet Label avec le texte 'Full name:'
label = ttk.Label(root, text='Full name:')

# Lier le texte à une variable
resultsContents = StringVar()
label['textvariable'] = resultsContents
resultsContents.set('New value to display')

# Afficher une image

# pip install pillow
# from PIL import ImageTk, Image
# myimg = ImageTk.PhotoImage(Image.open('myimage.png'))
image = PhotoImage(file='myimage.gif')
label['image'] = image
```

Fin

Fin, End